

Strategy Design for a Top-Performing Toss or Hold'em Pokerbot (Pokerbots 2026)

Executive Summary

- **Exploitative Focus:** Prioritize a highly exploitative strategy that adapts to opponents' tendencies, rather than a static Game-Theoretic Optimal (GTO) approach. Equilibrium-based bots are safe but *lack adaptability*, always playing the same way regardless of opponent – not optimal when facing suboptimal strategies ¹. An exploitation-focused bot can achieve *greater rewards* by targeting specific opponent weaknesses ², as seen in research where adaptive agents greatly outperformed static Nash-based bots ³. In practice, pure GTO bots “play not to lose” and often underperform in profit ⁴.
- **Viable Strategy Stacks:** Three primary strategy paradigms can be considered (see comparison table below): (1) **Abstraction-based** – solving a simplified model of the game (e.g. via CFR or deep RL) to serve as a strong baseline policy; (2) **Opponent Modeling** – dynamically learning the opponent's style and adjusting to exploit their specific leaks; and (3) **Equity/Heuristic-based** – using precomputed equity tables, hand strength evaluation, and rule-based logic (possibly with PokerStove) to make decisions. A hybrid approach combining these (e.g. a GTO-ish baseline with exploitative overrides) yields a robust yet maximally punishing strategy.
- **Impact of Discard Mechanic:** *Toss or Hold'em* adds a unique flop discard round that fundamentally alters hand values and information flow. Each player starts with 3 hole cards and must discard one to the board on the flop ⁵. The discarded cards become *shared* board cards (public), meaning you retain access to your tossed card at showdown but so does your opponent ⁶. This results in 6 board cards by showdown instead of 5, making stronger hands (flushes, straights, full houses) more common. As in other multi-hole variants (e.g. Pineapple), *winning hands tend to be much stronger on average* ⁷, so strategies must tighten value thresholds and emphasize nut draws. The discard choice itself is a rich source of information – which card an opponent throws can signal their hand's composition or what draws they value, creating an extra opportunity to exploit their decision.
- **Fast Equity Estimation:** With no time to “ponder” off-turn, the bot must evaluate hand equities and outcomes extremely quickly on its turn. Incorporate an optimized equity calculator or precomputed lookup tables (LUTs) for hand strength. For example, PokerStove's equity engine (if allowed) can be integrated or used offline to generate tables of win probabilities for various hand/board scenarios ⁸. Efficient bitwise hand evaluators or neural approximators can compute win odds in microseconds by referencing precomputed values for 5-7 card combinations. This ensures the bot can compare options (fold/call/raise or which card to discard) based on *expected value (EV)* without expensive real-time simulation. Experienced bot developers note that having a fast equity calc “makes everything easier,” enabling equity-based decisions without complex logic ⁹.

- **Online Opponent Modeling:** Over a 1000-hand match, even subtle opponent patterns become statistically significant. The bot should track the *most informative opponent behaviors*, such as **discard decisions** (what type of card do they tend to toss – e.g. lowest kicker, unsuited card, etc.), **bet sizing tendencies** (do they overbet only with strong hands or bluffs? do they min-bet draws?), and **board texture interactions** (e.g. check-folding on scary boards vs bluffing them). These observations feed into an opponent model updated as the match progresses. Effective opponent modeling in poker is *critical to high-level play*, since exploiting an opponent's specific mistakes yields far more profit than any one-size-fits-all strategy ¹⁰ ¹¹. The model can be as simple as statistical frequencies (e.g. opponent folds to 3-bet 70% -> over-bluff them) or as complex as a Bayesian belief over their hole cards updated each action. Key is to identify **consistent leaks** – for example, if their discard reveals they always keep high cards, you infer they often throw away low connectors (which means their post-discard range is more high-card heavy, informing your bluffing and calling decisions). By learning an opponent's profile (loose/tight, passive/aggressive, draw-heavy, etc.) within the first few hundred hands, the bot can *dynamically adjust its strategy* to maximize exploitation of that profile ¹².

- **Discard Strategy Optimization:** The discard round introduces a strategic decision point unfamiliar to standard Hold'em. A top bot should have a dedicated module or heuristic for **choosing which card to discard** to maximize its final hand equity. Because the discard becomes a public card, the bot must weigh not only which two cards give *it* the best 5-card hand, but also how giving that card to the board might benefit the opponent. *Optimal discard selection can be approached by brute-force equity analysis*: compute the win probability of each discard option (keeping different combinations of two hole cards) against a range of likely opponent holdings, given the flop. Then choose the discard that yields the highest EV for the bot's hand. For instance, if discarding a certain card would complete a flush or straight on the board that the opponent could also use, it might be wiser to discard a different card unless you are also benefiting equally. In the example from the rules, the out-of-position player discarded the J♥ on a two-heart flop, inadvertently *giving the in-position player a flush* ¹³. A strong bot would avoid such mistakes by evaluating that *keeping* the heart (and perhaps discarding a different card) was better to deny the opponent an immediate flush. In general, **keep private the cards that give you a unique advantage** (e.g. a pocket pair or a suited card that could complete a draw) and discard the card that *adds the least value or the most risk* when made public. The bot's discard logic can be pre-trained through simulations: e.g. enumerate many flop scenarios and determine which discard choice leads to the highest win rate for each type of 3-card hand. Over time, this builds a reliable heuristic or table for discard decisions under various board textures and hand compositions.

- **Long-Term Exploitation vs Risk Management:** With 1000 hands per match, the bot has ample runway to exploit an opponent, but it must also manage risk to avoid wild bankroll swings. A highly exploitative style can yield high average profits but might be more variance-prone or temporarily exploitable itself. The strategy should **balance immediate exploitation with long-term stability**. In practice, this means two things: (1) **Adaptive aggression** – ramp up exploitative deviations gradually as confidence in the opponent model grows. In early hands, use a mixed or moderately conservative strategy until you have enough data to be sure of a read (avoiding big gambles on a hunch). As the opponent's leaks become clearer, increase pressure in those spots to maximize EV. (2) **Minimize catastrophic risk** – because each hand resets with 400 chips, losing one all-in is not match-ending, but consistently high-variance play could backfire. The bot can incorporate bankroll-aware decisions; for example, if it's far ahead in the match, it can shift to lower-variance play to protect the lead,

forcing the opponent to take risks instead. Conversely, if behind, it may take calculated high-variance shots (e.g. more willing to coin-flip with slight edge) to try to catch up. Overall, over 1000 hands a superior strategy will *win by a thousand cuts* – reliably exploiting small mistakes hand after hand – rather than relying on a few huge coinflip pots.

- **Efficient Action Selection:** The bot must **make decisions under strict time and computation constraints** (no thinking on opponent's turn, and presumably a short time limit on each action). To meet this, implement *staged computation and caching*: break complex decision logic into stages that can return a good-enough action quickly if needed. For example, first run a fast heuristic evaluation (using cached results or a simple lookup) to narrow down candidate moves, then if time permits, run a deeper simulation or search for the best move among those candidates. Always ensure there's an "early exit" – if one option is clearly superior by a large margin in preliminary analysis, take it without fully simulating every alternative. **Caching** can dramatically speed up repeated calculations: store and reuse results for situations likely to recur. This could include caching equity calculations for certain common board textures, or reusing the opponent model's parameters from previous similar states. Since the game engine resets each hand, caching is most useful *within a hand* (e.g. reusing a Monte Carlo roll-out result for a similar situation on a later street) or across hands for opponent tendencies. Additionally, ensure the bot uses **time management** – e.g. allocate more computation to high-stakes decisions (big pot or all-in call) and use quick default decisions for trivial spots. By structuring the algorithm as an anytime process (where it can return the best found action at any point if the timer nearly expires), the bot guarantees it will act within the time limit. This prevents losses due to timeouts and keeps the strategy responsive.
- **Robust Testing and Deployment:** Plan a thorough offline testing regimen to validate the bot's performance and resilience before competition. This includes extensive **self-play** (to ensure the strategy doesn't have obvious holes and that the opponent modeling logic doesn't behave pathologically against itself) and **versus fixed benchmark agents**. Test against a spectrum of opponent styles: a tight passive bot, an aggressive bluffing bot, a random-action bot, and any known bots from previous competitions. These evaluations will reveal if the bot properly exploits the weak opponents and at least breaks even against tougher or unpredictable ones. Use the results to iteratively refine the strategy – e.g. if the bot fails to adjust to a shifting opponent strategy, improve the adaptation logic. Moreover, simulate full 1000-hand matches to see how bankroll evolves and ensure variance is within acceptable range. **Security and reliability** testing is also crucial: the final bot should run consistently without crashes, memory leaks, or rule violations under competition conditions. Check that it doesn't log or output any sensitive data (to avoid giving away strategy or violating rules) and that any machine learning components do not inadvertently carry over information between matches (no data leakage). Finally, verify **environment compatibility** – the bot must run on Python 3.7 with the given engine API and respect the <100MB size limit. This may involve trimming dependencies (e.g. using a lightweight neural net library or freezing the model to a raw numpy array) and static-linking any compiled code for hand evaluation (to avoid library conflicts). By covering these bases, the bot will be primed for a smooth deployment.

Strategy Stack Options Comparison

Strategy Approach	Technique & Key Idea	Pros	Cons
Abstraction-Based (GTO)	Solve a simplified Toss or Hold'em model offline (e.g. via CFR or deep RL) to generate a near-equilibrium policy. Use state and action abstraction to reduce the game size, then play according to this fixed strategy.	– Robust vs. any opponent: Unexploitable by design; avoids making major mistakes.
 – Proven vs strong players: Equilibrium-based agents have beaten pros in standard Hold'em ¹⁴ ¹⁵.
 – Clear baseline: Provides a solid starting strategy that can be augmented with exploits.	– No opponent adaptation: Ignores opponent-specific weaknesses, leading to lower win-rate against weak players ¹.
 – Resource heavy: Computing and storing an approximate solution for this large variant is computationally intense.
 – Predictable: Plays the same way every hand, which an adaptive opponent could learn to counter (if they deviate in a way that the equilibrium doesn't punish).

| **Opponent Modeling Exploiter** | Build an online model of the opponent's behavior (e.g. frequencies of bets, discard choices, range of hands shown) and derive counter-strategies on the fly. For example, use Bayesian updates or machine learning (like a lightweight neural net or regression) to predict opponent actions and hand strength, then choose the best counter-action (a best response) for the current opponent model. | – **Maximizes profits:** Targets specific opponent leaks to extract extra EV, often yielding higher rewards than any static strategy ².
 – **Adaptive:** Improves over time within the match; can adjust if the opponent changes strategy.
 – **Effective vs. common styles:** Quickly exploits predictable patterns (e.g. over-folding or over-bluffing) that many bots will have, turning their mistakes into our gains ¹⁰. | – **Model risk:** If the opponent model is inaccurate or the opponent is unpredictable, the bot might make costly missteps (e.g. calling bluffs that aren't there).
 – **Complexity:** Implementing real-time learning and ensuring stability (not overfitting to noise) is challenging under runtime constraints.
 – **Exploitable windows:** During the learning phase, or if the opponent detects our counter-strategy, our bot could itself be exploited (e.g. opponent shifts strategy to counter our expected response). |

| **Equity & Heuristic (LUT-Based)** | Rely on precomputed hand equity values and simple heuristics for decisions. For example, use a table of win probabilities for every possible hand vs random or vs an estimated opponent range (computed via PokerStove or offline simulation). Decisions (bet, call, fold, or discard choice) are then made by comparing equity to thresholds (like pot odds or bluff frequency). Hand strength metrics like *effective hand strength (EHS)* can be used to simplify state evaluation. | – **Fast and lightweight:** Lookup tables and straightforward calculations are extremely fast at runtime – crucial under Python 3.7 and no pondering. Minimal CPU and memory (fits in <1GB easily) ⁹.
 – **Simplicity = reliability:** Fewer moving parts means fewer bugs; easy to debug and test.
 – **Baseline exploitability:** Even a basic equity strategy will play reasonably and can be tuned to exploit obvious imbalances (e.g. call more when equity is high vs a bluff-happy opponent). | – **Limited sophistication:** Heuristics might not capture complex strategic concepts (e.g. the value of blockers, or multi-street game plans).
 – **Requires tuning:** The fixed thresholds (for calling or betting) might need adjustment for Toss or Hold'em's dynamics, and could misfire if the opponent's style diverges from assumptions.
 – **Static behavior:** Without

deeper opponent modeling, it reacts only to immediate equity; a clever opponent could predict our thresholds and exploit them (e.g. bluff exactly when our rules say to fold). |

| **Search-Based (MCTS/RL Policy)** | Utilize real-time simulation or a pre-trained neural network policy for decision-making. A Monte Carlo Tree Search (MCTS) could simulate possible action sequences from the current state, using a model of opponent behavior (e.g. assume opponent follows some strategy profile) to estimate outcomes ¹⁶. Alternatively, train a neural network via self-play reinforcement learning specialized for Toss or Hold'em, and use it as the bot's policy (with slight online fine-tuning if allowed). | - **Lookahead capability:** Simulating ahead can account for future card outcomes and betting rounds, potentially finding more accurate decisions than static evaluations.
 - **Can encode complex strategy:** An RL-trained policy or value network can implicitly learn nuanced strategies (bluffs, traps, etc.) from large-scale self-play.
 - **Balanced style:** Tends to produce a strong overall strategy (approaching GTO) with the ability to bias towards exploitation if the opponent model in MCTS is tuned to the current foe. | - **Computational load:** MCTS or deep networks are heavy – they risk exceeding time or memory limits. Without substantial optimization (or a smaller game tree abstraction), real-time search may not finish in time.
 - **Large code size:** Neural net frameworks or big models can threaten the 100MB size cap. Compression or custom inference code is needed, adding development overhead.
 - **Uncertain behavior:** A learned policy is a black box – it might have failure modes that are hard to diagnose, and if the opponent is far outside its training distribution, it may not adapt well. |

Table: Comparison of strategy approaches for building a Toss or Hold'em pokerbot. A hybrid strategy can combine a baseline equilibrium approach with opponent modeling and fast equity calculations to get the best of all worlds.

Recommended Strategy and Justification

Adopt a Hybrid Exploitative Strategy: We recommend a **stacked approach** that layers an unexploitable baseline with aggressive opponent-specific exploitation. In practice, this means the bot will **start with a sound foundational strategy** (using a simplified game solution or well-tuned heuristic policy as a default), and then **continuously adjust its play by learning from the opponent's behavior**. This hybrid approach leverages the strengths of all three strategy paradigms:

- **Baseline Module (Safety Net):** An opening book or baseline policy derived from offline computation ensures the bot plays *reasonable poker even with no reads*. For example, using a solved abstraction for Toss or Hold'em preflop and common flop scenarios gives the bot a GTO-like backbone to fall back on. This prevents the bot from making egregiously bad plays early on or in unfamiliar situations – essentially a form of **risk management** so that pure exploitation doesn't lead to self-destruction. As studies note, equilibrium strategies are good insurance: they guarantee at least break-even performance ¹¹. Our bot will use the baseline for the first few rounds and in any spot where the opponent model's confidence is low. This keeps it *competent against any opponent* and buys time to gather data.
- **Opponent Modeling & Exploitation Module:** On top of the baseline, the bot runs an **opponent modeling process** that feeds on every action the opponent takes – their bets, timing (if available), discard choice, showdown hands, etc. The model could be a simple set of statistical counters (e.g. how often they bluff on the river, how often they discard a high card) or a more sophisticated supervised learning model that classifies the opponent's style. Given the 1GB RAM and 100MB size

limit, a lightweight approach is advisable: for instance, maintain running frequencies and use a regression formula or shallow neural network to predict opponent tendencies. Every decision, the bot updates this model and **derives an adjusted strategy biased to exploit the opponent**. If the opponent folds too often, the bot will start bluffing more aggressively; if they never fold, the bot shifts to value-betting only. If we detect the opponent always discards their lowest card, we deduce they prefer high cards – the bot can then widen bluffs on low boards, etc. The exploitation module essentially computes a *best-response* to the opponent model at each juncture – this aligns with the principle that to maximize reward, one should play the strategy that best exploits the specific opponent ¹². Crucially, we implement a smooth blending between the baseline and the exploit strategy. For example, we might use a parameter α (starting near 0) that increases over time, where the final action chosen is $\text{Action} = \arg\max(\alpha \cdot \text{EV}_{\text{exploit}} + (1-\alpha) \cdot \text{EV}_{\text{baseline}})$. Early on, α is small (stick closer to baseline), but as confidence in reads grows, α to 1 (fully exploitative). This approach guards against early misreads and then ramps up aggression when it's statistically justified – effectively balancing exploitation with reliability over 1000 hands.

- **Discard Decision Engine:** We give special attention to the **flop discard round**, as it's a one-time chance each hand to either gain an edge or concede one. The bot will use an **EV-maximizing discard strategy**: enumerate the three possible hole-card pairs it could end up with after discarding, and evaluate each pair's winning chances given the visible cards. For the out-of-position player (who must discard first), this evaluation is against *unknown* opponent cards, so the bot assumes a plausible range (likely the opponent's preflop range, since no actions have occurred yet). For the in-position player (who discards second), the evaluation is against the opponent's *post-discard range*, which can be inferred partly from the card the opponent just discarded (and any betting preflop). In both cases, the bot effectively asks: *"If I discard card X, what is my expected equity (or final hand strength distribution) versus the opponent's range?"* Fast equity calculations come into play here. By leveraging a precomputed LUT or a cached simulation for the given flop+discard scenario, the bot can get these equity estimates on the fly. It then discards the card that yields the highest equity outcome. This algorithmic approach ensures the bot makes near-optimal discard choices, rather than relying on a static rule. It will automatically handle nuances like discarding to block the opponent: for example, if the opponent's discard made a flush possible on board, the bot's equity calculator might show that keeping a card of that flush suit (so as not to put another of that suit on board) preserves a higher win chance. In essence, the bot "thinks through" the discard: it keeps the two cards that best maximize its winning probability *given that the third card will now help both players*. This level of calculation is feasible within constraints because it's a one-time decision (only 3 options to consider) and can be heavily optimized with precomputed outcomes.

- **Fast Equity and Simulation Tools:** Throughout decision-making, our strategy leans on **efficient equity evaluation** for speed. We integrate a hand evaluation library (possibly a Python C extension using bit-level hand rankings or a port of PokerStove's engine). This allows instant computation of win percentages for any given hand matchup or board scenario. For example, on the river the bot can exactly calculate its hand's win probability against the opponent's modeled range by enumerating possible opponent hands that fit the observed actions. On earlier streets, it can compute *expected value of betting* by simulating a few likely outcomes (turn/river draws). We can also deploy **Monte Carlo rollouts** in critical spots: e.g. facing an all-in, simulate random deals of the remaining cards to estimate equity. Because we disallow thinking on the opponent's turn, any heavy simulation must occur in our turn time window – thus, we limit Monte Carlo depth and count.

Instead, many simulations are done **offline**. Prior to the competition, we will have generated tables for common scenarios (like typical flop textures and hand ranges) so that in-game the bot mostly does table lookups or very shallow simulations. We also consider a **neural network approximator** for flop/turn situations: a small neural net (with inputs encoding our hand, board, and some opponent model stats) could output an approximate win probability or optimal action. This network would be trained on data from self-play or solved subgames. If used, we'll compress it (e.g. 32-bit floats to 8-bit) to save space. Importantly, all these computation aids are geared to fit in memory and time limits – e.g., a hand rank LUT for all 7-card combinations is ~133 million entries, which is too large, but using a perfected algorithm like Texas Hold'em *hand rank bitboards* or lookup by prime product can evaluate a hand in a few CPU instructions. The bottom line is **every decision will be backed by a numeric evaluation (equity, EV or lookup)** rather than pure intuition, ensuring the bot is calculating profitable plays rapidly and consistently.

- **Time Management & Action Selection:** The recommended bot architecture uses a **hierarchical decision process** that guarantees timely actions. At the top level, for any decision (bet/call/fold or how much to bet), the bot considers a limited set of candidate actions: e.g. fold, call, or a few bet sizes (perhaps predetermined sizes like half-pot, full-pot, or all-in). This action abstraction keeps the decision space small. Next, for each candidate, the bot does a *bounded evaluation*: this could be a one-step lookahead (calculate immediate EV if that action is taken, assuming opponent responds in a typical way) or using the opponent model to predict opponent's likely response and resulting outcome. Because the game is zero-sum, the best action maximizes our expected chip gain. The bot will pick the action with the highest evaluated EV. If at any point the bot runs low on time (say the evaluation of one action is taking too long), it will employ a fail-safe: either fall back to the baseline action or use the partially evaluated results to pick an action. We will implement this with Python's time library or a move timeout from the engine – for example, ensure no search iteration runs beyond e.g. 90% of the allotted time. We also cache results within a hand: if we already computed win probabilities for a similar state (e.g. same board texture and similar betting context), reuse it instead of recomputing. The **no pondering constraint** means the bot can't pre-compute during opponent's turn, but we can make up for that by doing as much as possible *between rounds* (if the engine allows some initialization) – e.g. pre-loading LUTs, or even simulating likely scenarios for the next street while the current street decision is being made (interleaving simulation with waiting for opponent if multi-threading were possible). However, to keep it simple and safe under rules, we'll stick to single-threaded computation on our turn only. By careful calibration (based on offline profiling), we'll ensure the bot's decision logic for most routine actions completes very quickly (millisecond range), leaving extra time for hard spots (like a big river all-in call calculation). This staged approach guarantees the bot **never times out** and always makes a legal action, even if that means sometimes defaulting to a reasonable baseline move when under extreme time pressure.
- **Adaptive Risk and Exploit Tuning:** Over 1000 hands, the bot has the freedom to occasionally deviate or try exploratory moves without jeopardizing the match. Our strategy includes an element of **exploration vs. exploitation** akin to multi-armed bandits: if the opponent model has low confidence about a certain opponent tendency, the bot might take a speculative action to gather information – for example, calling a moderate bet with a marginal hand to see the opponent's cards at showdown, thereby learning how they play certain holdings. The slight cost of one thin call can be outweighed by the information gained, which could be leveraged in later hands. This is analogous to "information probing". Conversely, once the bot is highly confident in a read (e.g. it has seen the opponent shove bluff 3 out of 3 times in a specific spot), it will press hard on that read (e.g. making

light calls in that spot going forward). The long match length allows these adaptations to pay off. Additionally, **match state awareness** is built-in: the bot tracks the cumulative score. If we are comfortably ahead in chips as the match nears the end, the strategy shifts to *minimize variance* – e.g. avoid calling all-ins without premium hands, and prefer safe plays that lock in the win. If we are behind, the bot may increase variance – e.g. more willing to play for stacks with semi-strong hands or to attempt a big bluff – since a controlled loss is no better than a wild attempt to win (second place in a head-to-head match is just losing). This dynamic approach to risk ensures the bot is always aligning its strategy with the ultimate goal (winning the match, not just individual hands). It's a form of *meta-strategy* that sits above the hand-by-hand tactics.

Justification: This multi-faceted strategy is justified by both theory and practical constraints:

- From a *theoretical standpoint*, combining a Nash-equilibrium baseline with opponent-specific adaptation is a well-known approach to achieve high performance. It's often called a *"two-tier" strategy*: first tier guarantees solidity, second tier seeks maximum exploitation ¹⁰. Academic work has confirmed that while pure equilibrium play is suboptimal in many real settings, an agent that can model and exploit opponents while falling back on a safe strategy when needed will dominate in the long run ² ⁴. Notably, the Alberta poker research group emphasized that *to achieve the highest level of play, one must model and exploit each opponent's tendencies* – an optimal strategy that ignores opponent weaknesses "would not maximize winnings against most typical opponents" ¹¹. Our approach embodies that philosophy.
- In terms of the *discard mechanic*, our strategy is explicitly designed to exploit it. Many competitors might treat the discard in a simplistic way (always toss the lowest card, etc.), which our bot can capitalize on. By actually computing the EV impact of each discard, our bot gains a strategic edge in those flop moments that can carry through the rest of the hand. Furthermore, by watching the opponent's discard decisions, our bot picks up reads that no normal Hold'em bot would have access to (since revealing a card is unusual in poker). This is a unique information asymmetry introduced by Toss or Hold'em, and our strategy is poised to make the most of it. For example, if the opponent frequently discards a suited card when a flush draw is present on the flop, it implies they don't value the draw – likely they have two better cards (maybe a pair or high cards). This insight could trigger an exploit like bluffing on flush scare cards because the opponent probably isn't on the flush. In short, **we turn the discard into an advantage**, not just a mechanical step.
- Considering *compute and memory constraints*, the recommended approach is feasible. The baseline strategy (if using abstraction or precomputed rules) can be stored in a compact form – e.g. a table of recommended actions for a coarse set of game states, or a small neural network policy. The opponent model stats are just a few dozen numbers in memory. The LUTs for equity can be large if done naively, but we will utilize efficient encoding (for instance, storing only needed slices of the equity table, or computing certain equities on the fly with bit operations). 1GB RAM is plenty for our needs – even an exhaustive preflop simulation table for 3-card vs 3-card equities would be at most a few hundred MB uncompressed, and that can be compressed under 100MB easily. Since the bot cannot rely on internet or external resources at runtime, we ensure *all necessary data and computation tools are baked into the bot*. This means including any C++ equity library in the package (statically linked) and verifying it runs on the target Python 3.7 environment. We favor Python libraries like NumPy (which is typically allowed and efficient in C) for heavy computations and will use vectorized operations whenever possible instead of pure Python loops, to stay within time limits.

- Our action selection logic is designed to be *modular and testable*. Each component (equity eval, opponent model update, discard solver, betting strategy) can be unit-tested in isolation using offline hand histories or simulation. This makes it easier to debug and optimize. Moreover, the modular design allows stripping out or simplifying pieces if performance is an issue. For example, if Monte Carlo tree search turns out too slow, we can dial it back to a one-step lookahead or none at all, relying more on the neural approximator or LUT. The strategy is not tied to one technique – it's flexible, which is important given the unpredictable nature of opponents.
- **Exploitative potential:** Ultimately, this strategy is aimed at *maximizing winrate against the field*. Many competition bots (especially from less experienced teams) might use straightforward strategies – e.g. always discard the lowest card, or use fixed betting routines. Our hybrid bot will detect these patterns and respond with tailored counter-strategies. If an opponent plays too conservatively, our bot will relentlessly steal pots. If another plays like a maniac, our bot will trap and call them down lighter. This adaptiveness means our performance against the **average** opponent will be far higher than a static bot's. As an example, in the 2017 competition a static balanced bot could be beaten by a simple exploitative method that raised every hand once it noticed the opponent folded too often. We generalize this idea: whatever the common weaknesses in 2026 (and we anticipate some, like poor discard usage or imbalanced aggression), our strategy is equipped to learn and punish them. Meanwhile, against the very best opponents (who might also adapt), our baseline ensures we don't get blown out – the match may be closer to a Nash equilibrium confrontation in those cases, which is acceptable. Since all matches count for the final standings, crushing the weak and drawing with the strong is a winning formula.

In summary, the recommended approach combines **pre-calculated strategic insight** with **live opponent-driven adjustments**, all executed through **fast, memory-efficient computations**. This gives our Toss or Hold'em pokerbot the highest chance of dominating the competition by exploiting each opponent's flaws while safeguarding itself against being exploited in return.

Implementation Considerations Checklist

- **Memory & Size Compliance:** Ensure the entire bot (code + data) stays under 100 MB compressed. Use binary data formats and compression for LUTs and trained models (e.g. compress tables with zlib, quantize neural network weights) to minimize footprint. At runtime, keep memory usage <1 GB by loading only necessary data into memory (for instance, load large tables on-demand or in slices). Test memory usage with Python profilers and by simulating long matches to catch any leaks.
- **Python 3.7 Compatibility:** Develop and test using Python 3.7, since the competition runtime is fixed at that version. Avoid using newer language features or libraries that require >3.7. For example, if using `dataclasses` (Python 3.7 supports them) or newer typing features, verify they work in 3.7. Pin all library versions to ones compatible with 3.7. If we use a virtual environment for development, match it to the deployment environment's OS (likely Linux) and Python version to catch platform-specific issues.
- **Integrated Equity Calculator:** Include a fast hand evaluation mechanism. Options include a Python extension (C/C++) for hand ranking. If we use PokerStove's equity calculator, integrate it by either calling its library functions or by precomputing and embedding its results. **Static link any C/C++ library** (like a modified PokerStove or open-source evaluator) to avoid dependency issues – the

compiled extension should work out-of-the-box when the bot is run. Confirm the extension loads properly in the sandbox environment. We'll perform a dry run on a clean environment to ensure no missing symbols or ABI mismatches occur. If using pure Python calculations for equity, use numpy for vectorized operations and possibly pre-generate lookup dictionaries for common cases (e.g. mapping 5-card hand to rank). The goal is every equity query returns in microseconds. *Verification:* Compare our equity outputs with known results from PokerStove or online calculators for correctness.

- **Opponent Model Data Handling:** All opponent modeling data must be kept in-memory (or serialized to disk only for our bot's internal use if permitted by rules). We need to ensure **no data leakage** across matches – e.g. if the bot writes opponent tendencies to a file for learning, that file should be unique per match or wiped afterward. However, most competitions disallow persistent storage between matches, so we will keep opponent profiles only in RAM during a match. Also, implement the model in a thread-safe/simple way (since we'll likely remain single-threaded) to avoid any concurrency issues. We'll include a function to reset the model state cleanly at the start of each match (important if the engine reuses the bot process for multiple opponents sequentially). Consistency and privacy are crucial: the bot should not reveal information about its own strategy or the opponent model through any side channel. This means no logging the opponent's patterns to stdout (unless debugging, and that will be removed in the final submission).
- **Time Budget Enforcement:** The bot's logic will be instrumented to respect time limits. Use timers or engine callbacks to measure how long each decision is taking. We will set a conservative deadline (e.g. if the engine allows 500ms per decision, we target completion in 300ms). Implement **early exits** in search loops: e.g., in MCTS or simulations, check elapsed time at each iteration and break if nearing the cutoff. Each decision function will have a failsafe to return a default action if the computation doesn't finish in time. We will test worst-case scenarios (complex game tree situations, maximal number of possible actions) to ensure even those complete on time or gracefully abort to a safe action. Caching results (like previously computed equities or game states) is part of this strategy to reduce repeated work. We'll also take advantage of the fact that Python execution can be the bottleneck – by pushing heavy calculations into C (numpy or our equity library), we get speed and also avoid GIL issues, since numpy releases the GIL on heavy ops.
- **Action Determinism & Randomization:** While exploitation implies deterministic responses to patterns, we may want some **randomization in action selection** to avoid being predictable (especially against adaptive opponents). For instance, even if we determine the optimal play is to bluff in a spot 80% of the time, we should introduce a random draw to actually bluff 80% and check 20%. We'll use a secure random generator (Python's `random.SystemRandom` or `numpy.random` with a seed set per match) to make these decisions. This randomness should be configured such that it does not affect reproducibility when needed (maybe allow seeding via an environment variable for debug). Importantly, ensure that any randomness does not cause divergent behavior that could leak information (e.g. do not base the random seed on any secret info like our hole cards – which could theoretically be exploited by an opponent if they had a way to guess our RNG state). Keep the random strategy components minimal and only where it's strategically justified (like mixing bluff frequencies). Most of the bot's logic will remain deterministic given the same game state and opponent model.

- **Error Handling & Stability:** The bot should never crash or throw an unhandled exception during a match. We will add robust error handling around critical sections – for example, if for some reason our equity calculator gets an illegal input or our model produces a NaN, catch it and fall back to a default action. All file I/O will be either removed or made failure-tolerant (in case file writing is not allowed or disk is read-only). Also, guard against potential issues like divide-by-zero (e.g. in pot odds calculations if pot size is zero), index errors in lookup tables, etc. By simulating a large number of random matches offline (including edge cases like extremely weird opponent actions), we can observe any runtime warnings or errors and fix them. Memory stability is also crucial – avoid structures that grow without bound. For instance, don't store every hand history in a list; instead, keep running averages or counters. Python's garbage collector should handle most, but we remain mindful of object lifetimes (e.g. reuse data structures each hand to avoid excessive allocation).
- **Testing in Simulation:** Prior to deployment, run the bot in the provided engine environment (or a mock thereof) in thousands of hand simulations. We can script matches between our bot and some dummy opponents (even if we have to write simple opponents ourselves) to ensure everything works in an actual game loop. This testing will cover things like: does the bot parse the game state correctly (cards, pot sizes, etc.), does it output actions in the correct format, does it handle the discard action timing properly (discarding when required), and does it remain within time limits over a long match. We should test scenarios such as: our bot on dealer vs opponent on dealer (to verify blinds and betting order logic), reaching showdown and tying (to see if any division by zero in bankroll update occurs), and extreme betting sequences (like multiple raises to all-in). These tests will also reveal any compatibility issues with Python 3.7 (e.g. if a library uses features not present, it will error out).
- **Security & Fair Play:** Confirm that the bot does not attempt any disallowed actions. This includes network calls (we will double-check that no part of our code tries to access the internet or external system – for example, if using a library that normally might check for updates, disable that). The bot should operate entirely within the information it's given by the engine – no screen-scraping or external communication (which is forbidden and not needed here). Also, ensure the bot's behavior is **consistent and not based on hidden state from previous matches**. Every match, we start fresh. If we use any machine learning components, we will not train them on the fly in a way that persists weights beyond the match (unless the competition explicitly allows learning across matches, which is rare and typically disallowed for fairness). Essentially, the bot's strategy at the start of a match against a given opponent should not depend on having played that opponent before. This avoids any accusations of meta-gaming or data leakage. In terms of *privacy of our strategy*, since our bot might incorporate clever exploits, we avoid exposing them. For example, if logging is enabled for debugging, we will strip out any logs that print our internal hand evaluations or opponent model assessments, because if such logs were accessible to opponents (say, on a shared server), it could reveal our tactics. We treat the competition environment as potentially observable and thus keep our cards (both literally and metaphorically) close to our chest.
- **Deployment & Compatibility:** Package the bot such that it's easy to deploy: typically a single entry-point script with all dependencies included. We will use relative imports or include modules in one directory to avoid import errors. If using compiled extensions, compile them for the correct platform (most likely Linux x64) and Python 3.7 ABI, and test on a clean machine. Documentation will be prepared (if required by organizers) to explain how to run the bot, but more importantly, the bot should run without special configuration. We will also consider using `pyinstaller` or similar to

create a single binary if allowed – but given the size limit, a well-organized source distribution might be better. Perform a final check on **random seed usage**, **floating-point consistency** (to ensure no platform-specific issues), and remove any non-determinism that isn't part of strategy (like iteration over Python dicts which is insertion-ordered in 3.7+, so it should be stable). Finally, be mindful of **third-party library licenses** – any code we include (like PokerStove's algorithms or an open-source hand evaluator) should be allowed under the competition rules. We'll attribute or remove any such code as needed. With all these considerations checked off, the bot will be ready to deploy with high confidence in both its performance and its compliance.

Bibliography

1. **MIT Pokerbots 2026 – Toss or Hold'em Rules (2026)**: Official variant description for the IAP 2026 Pokerbots competition, detailing the Toss or Hold'em gameplay (3 hole cards, flop discard round, etc.) ¹⁷ ⁶ .
2. **Adams, A. – Home Game Heroes: Strategy With Extra Hole Cards (PokerNews, 2015)**: Article discussing Pineapple and related Hold'em variants where players have extra hole cards and must discard. Highlights that extra cards produce stronger hands and suggests strategy adjustments (play more draws, beware of non-nut hands) ⁷ ¹⁸ .
3. **Li, X. & Miikkulainen, R. – Dynamic Adaptation and Opponent Exploitation in Poker (GECCO 2018)**: Research paper presenting an evolutionary opponent modeling approach. It demonstrates that exploiting opponent weaknesses yields higher utility than equilibrium play, defeating a top static poker AI (Slumbot) by adapting to new opponents ³ ¹⁹ .
4. **Xu, J. et al. – Efficient Opponent Exploitation in No-Limit Texas Hold'em (Electronics, vol.10, 2021)**: A study combining neuroevolution and reinforcement learning for poker. It emphasizes the limitations of equilibrium bots (no adaptation) and notes that targeted opponent exploitation can achieve greater rewards by maximizing utility against the current opponent ¹ ² .
5. **Davidson, A. et al. – Improved Opponent Modeling in Poker (Univ. of Alberta, 2000)**: Classic paper on poker AI highlighting the importance of modeling each opponent's style. Argues that an optimal (GTO) strategy "would not maximize winnings against most typical opponents" and that adaptive strategies are critical for expert-level play ²⁰ ¹¹ .
6. **Van den Broeck, G. et al. – Monte-Carlo Tree Search in Poker using Expected Reward Distributions (ACML 2009)**: Introduces an MCTS approach for no-limit poker. Distinguishes between Nash-based bots and exploiting bots, and proposes using opponent modeling (via regression trees) inside search to exploit weaknesses ²¹ ¹⁶ .
7. **Stack Exchange (Poker) – "How to get started building a poker bot?" (2017)**: Community Q&A where user "paparazzo" advises building an equity calculator and using an exploitative (non-GTO) strategy. Notably states "GTO bot was a disaster... GTO you are playing not to lose," recommending an equity-driven, opponent-exploit approach instead ⁹ ⁴ .

8. **ThePokerBank – PokerStove Equity Calculator Review:** Guide explaining PokerStove's functionality as an equity calculator for poker. Emphasizes that calculating hand equity versus ranges is fundamental to decision-making (e.g. comparing win% to pot odds to decide on calls) ⁸ ²² .
9. **MIT Pokerbots Example (2026) – Toss Round Scenario:** An illustrative example from the Toss or Hold'em rules where one player's discard gives the opponent a flush ¹³ . This highlights the strategic danger of discarding a card that significantly strengthens the shared board (and thus the opponent's potential hand).
-

¹ ² ¹² ¹⁵ Efficient Opponent Exploitation in No-Limit Texas Hold'em Poker: A Neuroevolutionary Method Combined with Reinforcement Learning | MDPI

<https://www.mdpi.com/2079-9292/10/17/2087>

³ ¹⁴ ¹⁹ Dynamic Adaptation and Opponent Exploitation in Computer Poker

<https://nn.cs.utexas.edu/downloads/papers/xun.gecco18.pdf>

⁴ ⁹ texas hold em - How to get started building a poker bot? - Poker Stack Exchange

<https://poker.stackexchange.com/questions/9172/how-to-get-started-building-a-poker-bot>

⁵ ⁶ ¹³ ¹⁷ variant.pdf

file:///file_00000000495071f4a6ef39deee569cc3

⁷ ¹⁸ Home Game Heroes: Strategy When Playing With Extra Hole Cards in Flop Games | PokerNews

<https://www.pokernews.com/strategy/home-game-heroes-strategy-extra-hole-cards-flop-games-22385.htm>

⁸ ²² PokerStove Review | Poker Stove Equity Calculator & Alternatives

<https://www.thepokerbank.com/tools/software/pokerstove/>

¹⁰ ¹¹ ²⁰ poker.cs.ualberta.ca

<https://poker.cs.ualberta.ca/publications/ICAI00.pdf>

¹⁶ ²¹ starai.cs.ucla.edu

<https://starai.cs.ucla.edu/papers/VdBBNAIC09.pdf>